# Data Type, Variable, and String

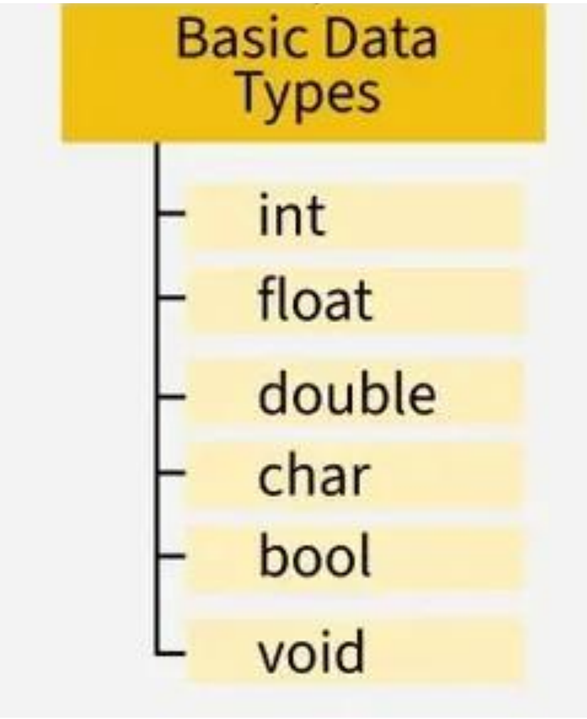
FEBRIAN WINSTON HUTAGALUNG, S.T., M.T.

# Outline

1. Data Type
2. Variable
3. String

# Data Type

- Each variable in C has an associated data type.
- It specifies the type of data that the variable can store like integer, character, floating, double, etc.




- int: Menyimpan bilangan bulat (tanpa desimal)
- float: Menyimpan bilangan desimal presisi tunggal (±6–7 digit signifikan)
- double: Menyimpan bilangan desimal presisi ganda (±15–16 digit signifikan)
- char: Menyimpan 1 karakter (huruf, angka, simbol). Contoh: char huruf = 'A';
- bool: Tipe logika dengan dua kemungkinan nilai: true (1) atau false (0). Contoh: bool status = true;
- void:  Tidak menyimpan nilai atau tidak ada tipe data.

# Data Type

- Each data type requires different amounts of memory and has some specific operations which can be performed over it.
- Different data types also have different ranges up to which they can store numbers. These ranges may vary from compiler to compiler.
- Format specifier adalah simbol khusus yang dipakai dalam fungsi printf() atau scanf() untuk menunjukkan tipe data yang akan ditampilkan atau dibaca.
- The next slide shows a table of data types and ranges in GCC.

# Data Type

| Data Type | Size (bytes) | Range | Format Specifier |
|---|---|---|---|
| char | 1 | -128 s/d 127 | %c |
| unsigned char | 1 | 0 s/d 255 | %c (%u angka) |
| short int | 2 | -32,768 s/d 32,767 | %hd |
| unsigned short int | 2 | 0 s/d 65,535 | %hu |
| int | 4 | -2,147,483,648 s/d 2,147,483,647 | %d |

# Data Type

| Data Type | Size (bytes) | Range | Format Specifier |
|---|---|---|---|
| unsigned int | 4 | 0 s/d 4,294,967,295 | %u |
| long int | 4 (32-bit) / 8 (64-bit) | 32-bit: sama dengan int / 64-bit: $-9.2 \times 10^{18}$ s/d $+9.2 \times 10^{18}$ | %ld |
| unsigned long int | 4 (32-bit) / 8 (64-bit) | 32-bit: sama dengan unsigned int / 64-bit: 0 s/d $1.8 \times 10^{19}$ | %lu |
| long long int | 8 | $-9.2 \times 10^{18}$ s/d $+9.2 \times 10^{18}$ | %lld |

# Data Type

| Data Type | Size (bytes) | Range | Format Specifier |
|---|---|---|---|
| unsigned long int | 8 | 0 s/d $1.8 \times 10^{19}$ | %llu |
| float | 4 | $\sim\pm3.4 \times 10^{-38}$ s/d $\pm3.4 \times 10^{38}$ | %f |
| double | 8 | $\sim\pm1.7 \times 10^{-308}$ s/d $\pm1.7 \times 10^{308}$ | %lf |
| long double | 16 | $\sim\pm3.4 \times 10^{-4932}$ s/d $\pm1.1 \times 10^{4932}$ | %Lf |
| bool | 1 | 1 atau 0 | %d |

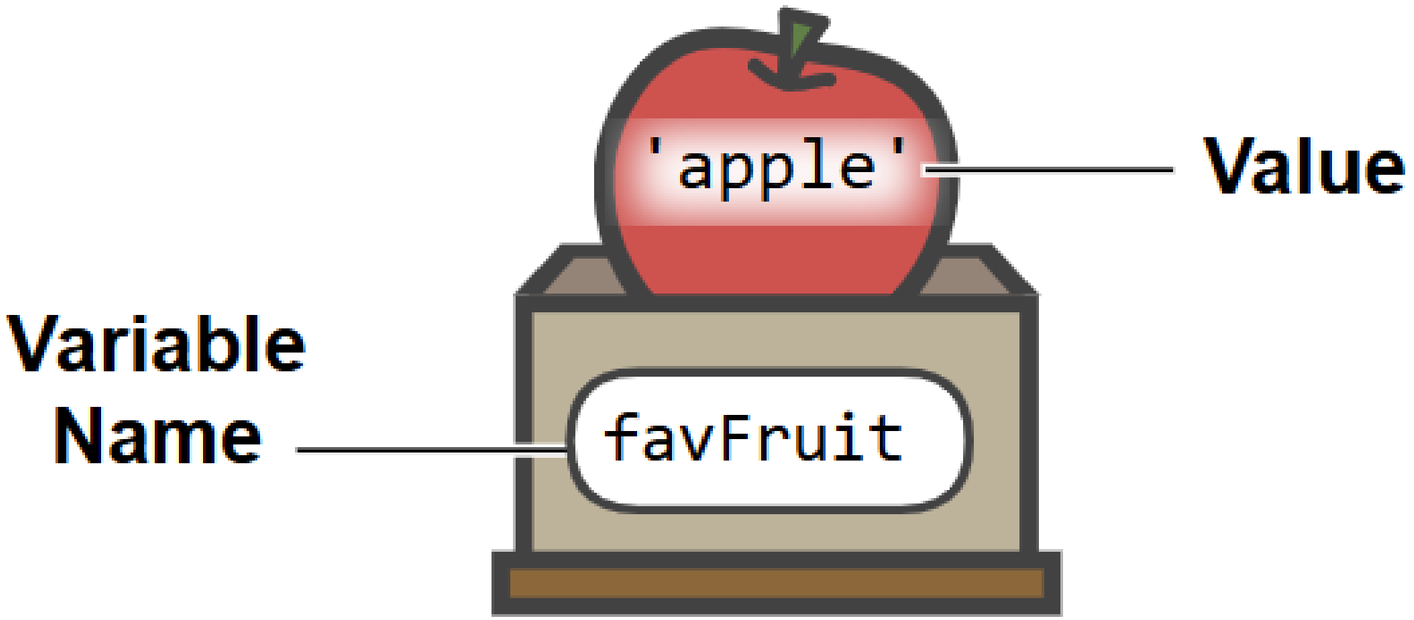
# Variable

- A variable in simple terms is a storage place that has some memory allocated to it.
- Basically, a variable is used to store some form of data.

**example:**

```
int umur = 20;

umur = 25;
```



# Variable

**Variable Declaration: A typical variable declaration is of the form:**

```
type variable_name;
or for multiple variables:
type variable1_name, variable2_name, variable3_name;
```

# Variable

A variable name can consist of alphabets (both upper and lower case), numbers, and the underscore ‘_’ character. However, the name must not start with a number.

```c
#include <stdio.h>

int main()
{
    int b = 28, c = 12;
    char huruf = 'K';
    printf("Character value : %c\n", huruf);
    printf("Integer value : %d\t%d\n", b, c);
    return 0;
}
```

# Variable

**Rules for defining variables:**

- **A variable can have alphabets, digits, and underscore.**
- **A variable name can start with the alphabet, and underscore only. It can’t start with a digit.**
- **No whitespace is allowed within the variable name.**
- **A variable name must not be any reserved word or keyword, e.g. int, etc.**

# Variable

**Difference between variable declaration and definition:**

- **Variable declaration refers to the part where a variable is first declared or introduced before its first use.**
- **A variable definition is a part where the variable is assigned a memory location and a value.**

# Variable

## Types of Variables in C

**Local variable: variabel yang dideklarasikan di dalam fungsi atau blok kode.**

**Ciri-cirinya:**

1. **Scope terbatas → hanya bisa digunakan di dalam fungsi/blok tempat ia dibuat.**
2. **Tidak bisa diakses dari luar fungsi/blok itu.**

```c
#include <stdio.h>

void function() {
    int x = 10;         // variabel lokal
}

// ini di luar function
printf("Nilai x: %d\n", x);
```

# Variable

## Types of Variables in C

**Local variable: variabel yang dideklarasikan di dalam fungsi atau blok kode.**

```c
#include <stdio.h>

void function() {
    int x = 10; // variabel lokal DIINISIALISASI
    printf("Nilai x: %d\n", x);
}

int main() {
    function();
    return 0;
}
```

```c
#include <stdio.h>

void function() {
    int x = 10; // variabel lokal DIINISIALISASI
}

int main() {
    printf("Nilai x: %d\n", x);
    return 0;
}
```

# Variable

## Types of Variables in C

**Global variable: Variabel yang dideklarasikan di luar semua fungsi atau blok.**

**Ciri-ciri:**

1. **Biasanya di awal program.**
2. **Bisa diakses oleh semua fungsi di file tersebut.**

```c
#include <stdio.h>
int x = 20;//global variable
void function1()
{
    printf("%d\n" , x);
}
void function2()
{
    printf("%d\n" , x);
}
int main() {

function1();
function2();
    return 0;
}
```

# Variable

## Types of Variables in C

- Static variable: Variabel yang menyimpan nilainya meskipun fungsi sudah selesai dijalankan. Dibuat dengan keyword static.
- Bedanya dengan local variable biasa:

1. Local variable → nilainya hilang setelah fungsi selesai.
2. Static variable → nilainya tetap tersimpan untuk pemanggilan berikutnya.

```c
#include <stdio.h>
void function(){
    int x = 20;//local variable
    static int y = 30;//static variable
    x = x + 10;
    y = y + 10;
    printf("\n%d,%d",x,y);
}
int main() {

function();
function();
function();
return 0;
}
```

output:

```
30, 40
30, 50
30, 60
```

# Variable

## Types of Variables in C

**Static variable: Variabel yang menyimpan nilainya meskipun fungsi sudah selesai dijalankan.**

**Pemanggilan pertama:**

- **x = 20 → x + 10 = 30**
- **y = 30 → y + 10 = 40**
- **Output: 30, 40**

**Pemanggilan kedua:**

- **x direset lagi ke 20**
  **→ x + 10 = 30**
- **y tidak direset**

**→ nilai sebelumnya = 40**

**→ y + 10 = 50**

- **Output: 30, 50**

**Pemanggilan ketiga:**

- **x direset lagi ke 20**
  **→ x + 10 = 30**
- **y = 50 → y + 10 = 60**
- **Output: 30, 60**

# Variable

## Types of Variables in C

**Automatic variable: Di bahasa C, semua variabel yang dibuat di dalam blok/fungsi secara default adalah automatic variable.**

**Disebut “automatic” karena:**

**1. Dibuat secara otomatis saat fungsi dipanggil.**

**2. Dihapus secara otomatis saat fungsi selesai.**

**Keyword auto bisa dipakai, tapi biasanya jarang ditulis karena default-nya memang auto.**

```c
#include <stdio.h>
void function()
{
int x=10;//local variable (also automatic)
auto int y=20;//automatic variable
}
int main() {

    function();
    return 0;
}
```

# Variable

## Types of Variables in C

- External variable:  variabel global yang bisa dipakai di lebih dari satu file C.
- Gunakan keyword extern untuk memberitahu compiler bahwa variabel itu didefinisikan di file lain.

**myfile.h**

```
1    extern int x;
```

**externfile.c**

```
1    #include "myfile.h"
2    int x = 19;
```

**Prak-2.c**

```
1    #include "myfile.h"
2    #include <stdio.h>
3    int main()
4    {
5        printf("Global variable: %d", x);
6    }
```

**output:**

```
D:\IT DEL\Pengajaran\2223 Ganjil\Daspro\Week 3\Prak>gcc Prak-2.c externfile.c -o prak2

D:\IT DEL\Pengajaran\2223 Ganjil\Daspro\Week 3\Prak>prak2
Global variable: 19
```

# Variable

## Types of Variables in C

**myfile.h**

```
1    extern int x;
```

deklarasi eksternal, compiler diberi tahu bahwa variabel x ada, tetapi definisinya bukan di sini.

**externfile.c**

```
1    #include "myfile.h"
2    int x = 19;
```

definisi variabel, x benar-benar dibuat di memori dan diberi nilai awal 19.

**Prak-2.c**

```
1    #include "myfile.h"
2    #include <stdio.h>
3    int main()
4    {
5        printf("Global variable: %d", x);
6    }
```

Program utama, karena sudah menyertakan deklarasi dari myfile.h, maka main() dapat mengakses x meskipun definisi aslinya berada di file lain (externfile.c).

# String

- Strings are defined as an array of characters.
- Declaration of strings: Declaring a string is as simple as declaring a one-dimensional array. Below is the basic syntax for declaring a string.

```
char str_name[size];
```

# String

## Cara Deklarasi String

```
1   char str1[] = "Hello";
2   char str2[10] = "World";
3   char str3[] = {'H','i','!','\0'};
```

- **str1 → compiler otomatis menambahkan \0 di akhir → "Hello\0"**



# String

## Cara Deklarasi String

```
1   char str1[] = "Hello";
2   char str2[10] = "World";
3   char str3[] = {'H','i','!','\0'};
```

- **str2 → panjang 10 karakter, tapi hanya dipakai sebagian (sisanya kosong).**



# String

## Cara Deklarasi String

```
1   char str1[] = "Hello";
2   char str2[10] = "World";
3   char str3[] = {'H','i','!','\0'};
```

- **str3 → tulis karakter satu per satu, lalu tambahkan \0.**



# String

## Read String

- You can use the scanf() function to read a string.
- The scanf() function reads the sequence of characters until it encounters whitespace (space, newline, tab, etc.).

main.c

Share Run

```c
#include <stdio.h>

int main() {
    char nama[20];
    printf("Masukkan nama: ");
    scanf("%s", nama);
    printf("Halo %s\n", nama);
    return 0;
}
```

Output

```
Masukkan nama: Andi Budi
Halo Andi


=== Code Execution Successful ===
```

# String

## Read String

- Dengan fgets(), string bisa mengandung spasi
- Membaca satu kalimat penuh

main.c

Share Run

```c
#include <stdio.h>

int main() {
    char nama[20];
    printf("Masukkan nama lengkap: ");
    fgets(nama, sizeof(nama), stdin);
    printf("Halo %s\n", nama);
    return 0;
}
```

Output

```
Masukkan nama lengkap: Andi Budi Doremi
Halo Andi Budi Doremi


=== Code Execution Successful ===
```

# String

## Read String

- Dengan fgets(), string bisa mengandung spasi
- Membaca satu kalimat penuh

main.c

Share Run

```c
#include <stdio.h>

int main() {
    char nama[50];
    printf("Masukkan nama lengkap: ");
    fgets(nama, sizeof(nama), stdin);
    printf("Halo ");
    puts(nama);
    return 0;
}
```

Output

```
Masukkan nama lengkap: Budi Doremi
Halo Budi Doremi


=== Code Execution Successful ===
```

# String

## Access String

- Since strings are actually arrays in C, you can access a string by referring to its index number inside square brackets [].

main.c

Share Run

```c
#include <stdio.h>

int main() {
    char nama[] = "Ani";
    printf("%c\n", nama[0]); // A
    printf("%c\n", nama[1]); // n
    printf("%c\n", nama[2]); // i
}
```

Output

```
A
n
i


=== Code Execution Successful ===
```

# String

## Modify String

- To change the value of a specific character in a string, refer to the index number, and use single quotes:

main.c

Share Run

```c
#include <stdio.h>

int main() {
    char nama[] = "Ani";
    nama[2] = 'a';
    printf("%s", nama);
}
```

Output

```
Ana

=== Code Execution Successful ===
```

# String

## Commonly used String Functions

- strlen(str) → Menghitung panjang string (tidak termasuk \0)
- strcpy(dest, src) → Menyalin isi string
- strcat(dest, src) → Menggabungkan string
- strcmp(str1, str2) → Membandingkan dua string
- strupr(str) → Ubah string ke huruf besar
- strlwr(str) → Ubah string ke huruf kecil

# String

## Commonly used String Functions

- **strlen(str) → Menghitung panjang string (tidak termasuk \0)**

main.c

Share Run

```c
#include <stdio.h>
#include <string.h>

int main() {
    char teks[] = "Halo";
    printf("Panjang string: %lu\n", strlen(teks));
    return 0;
}
```

Output

```
Panjang string: 4


=== Code Execution Successful ===
```

# String

## Commonly used String Functions

- **strcpy(dest, src) → Menyalin isi string**

main.c

Share Run

```c
#include <stdio.h>
#include <string.h>

int main() {
    char asal[] = "C Programming";
    char salin[50];
    strcpy(salin, asal);
    printf("Hasil salinan: %s\n", salin);
    return 0;
}
```

Output

```
Hasil salinan: C Programming


=== Code Execution Successful ===
```

# String

## Commonly used String Functions

- **strcat(dest, src) → Menggabungkan string**

main.c

Share Run

```c
#include <stdio.h>
#include <string.h>

int main() {
    char s1[50] = "Halo, ";
    char s2[] = "Dunia!";
    strcat(s1, s2);
    printf("Gabungan: %s\n", s1);
    return 0;
}
```

Output

```
Gabungan: Halo, Dunia!


=== Code Execution Successful ===
```

# THANK YOU!